
status: accepted date: 2026-05-17 tags: [memory, archivist, runtime-activation, write-path, complete] supersedes: [] depends-on: [ADR-0180, ADR-0181] implements: []

Memory Write-Path Unification

Context and Problem Statement

ADR-0181 declared strict exit at patch.143 (669/0/9, all 7 phases ) and the [§Closure plan amendment](#) sequenced the remaining program. The amendment's critical path (A1→A2→A3) is the cli read flip blocking task #100 — twice attempted, twice reverted with the same 5 deterministic acceptance failures (`adr0069-bug3-persist` , `p8-inv12-mem-full` , `e2e-0059-mem-search` , `e2e-0059-p3-unified-both` , `e2e-0059-p3-dedup`). The trace investigation (handover §root-cause, 2026-05-17) identified the cause as **incomplete Phase 5 on the write side**, not a flaw in the read handlers:

- Phase 5's exit gate language was "every MCP tool + CLI write command + hook + daemon routes through the archivist." Phase 5 delivered "100+ cli mcp-tool flips" — which covered MCP-mediated writes but missed the cli's internal write router.
- `routeMemoryOp({type: 'store'})` (`forks/ruflo cli/src/memory/memory-router.ts:1065-1077`) is the cli's internal write router used by `cli memory store` , `session_restore` , and every CLI-command write path. It stayed on the legacy direct-storage path.
- MCP writes therefore produce the archivist's rich-meta on-disk shape `{meta: {namespace, key, content, tags, ttl, ...}, ...}` ; CLI-command writes produce a flat shape with `namespace / key / content` at top-level and empty `meta` .
- The post-#99 dispatched reads (`handlers/memory/{get,list,search,search-unified}.ts`) project against the rich shape — fine for archivist-written records, broken for `routeMemoryOp` -written records. Re-flipping the reads without unifying the writes cannot converge.

The intent-fix commits retained from task #100 attempt 2 (`forks/agentdb b91b4fd` + `forks/ruflo 0eacaf6ec`) are forward-compat (correct API shape for `EmbeddingScorer.embed(text, {intent})`) but do not activate in production today and do not address this root cause.

This is a distinct decision because (a) it requires its own write-path audit + transition-period design that ADR-0181's amendments cannot cleanly absorb without further bloat, and (b) the dual-schema discriminator + v1 deprecation is a forward-compat surface that wants its own life-cycle tracking. Holding it in ADR-0181 would keep that ADR's "what is left" boundary blurred indefinitely.

Decision Drivers

- `feedback-data-loss-zero-tolerance` — 5 deterministic failures on memory round-trip is non-shippable. Either 100% or not fixed.
- `feedback-no-fallbacks` — a dual-schema read MUST NOT become a silent catch-and-continue. It is a documented transition surface, gated on a versioned discriminator, with v1 sunset tracked.

- `feedback-trace-before-hypothesis` — the trace identified the actual write/read asymmetry; the closure plan implements the fix, it does not re-hypothesise.
- `feedback-all-test-levels` — every step gates on unit + integration + acceptance.
- Phase 5's stated exit gate ("every MCP tool + CLI write command + hook + daemon routes through the archivist") is the contract this ADR completes for memory writes.
- ADR-0181's downstream closure path (Phase B memory-read handler readiness, Phase G bench re-baseline, Phase H heavy-skip review) gates on the cli read flip landing.

Considered Options

- **A. Write-path unification + dual-schema reads + cli read flip (chosen).** Flip `routeMemoryOp({type: 'store'})` through `archivist.dispatch('memory_store')` so both production write paths produce identical on-disk shape. Add a `shape_version: 1 | 2` discriminator on the read handlers so legacy on-disk records (written under v1) round-trip correctly during the transition. Then flip the cli reads. v1 sunset tracked in a separate future ADR.
- **B. Read-side coercion only.** Leave the two write paths divergent; teach each dispatched read handler to project from both shapes via field-presence checks (no versioning). Cheaper today.
- **C. Defer the cli read flip indefinitely.** Keep the handler bodies ready (already landed via #99) but never dispatch through them. CF#4 + CF#8 stay open in ADR-0181.
- **D. One-shot migration script.** Migrate every on-disk memory record to the rich shape in a single stop-the-world pass, then flip writes and reads cleanly with no transition period.

Decision Outcome

Chosen: **Option A**, because it is the only option that (1) honours Phase 5's stated exit gate for memory writes, (2) closes ADR-0181's task #100 / CF#4 / CF#8 / B / G / H gating, (3) keeps the dual-schema period bounded via the versioned discriminator + a tracked deprecation ADR, and (4) does not require a stop-the-world migration of production on-disk records.

Option B is rejected because it permanently divergent-fies the write paths — every future memory-read handler must carry the dispatched-vs-legacy shape knowledge, with no convergence path. It is exactly the unbounded compatibility surface that `feedback-no-fallbacks` warns against, dressed up as a feature.

Option C is rejected because it leaves the archivist's memory-read coverage scaffold-only despite the handler bodies being ready, and it leaves task #100 + CF#4 + CF#8 permanently open in ADR-0181 — a status that contradicts the ADR-0181 closure-plan amendment's "definition of done."

Option D is rejected because stop-the-world data migration on memory records is a class of risk this project has never run (memory state is the trust boundary for ReasoningBank + skill library + session restore), and the dual-schema approach achieves the same correctness with no migration window. If a migration is ever desired (e.g. to retire shape v1 cleanly), it is a follow-on ADR to this one, not a prerequisite.

Consequences

- Good, because Phase 5's exit gate is honoured for memory writes — the cli's internal write router joins the MCP boundary in dispatching through the archivist.
- Good, because ADR-0181's downstream closure path (Phase B + G + H) unblocks the moment A3 lands.

- Good, because the dual-schema is bounded — `shape_version: 1 | 2` is a versioned discriminator, not field-presence sniffing; v1 sunset is tracked in its own future ADR.
- Good, because no on-disk migration is required — existing records keep working as v1, new writes produce v2.
- Good, because `feedback-no-fallbacks` is honoured — records matching neither v1 nor v2 fail loud, not silently coerce.
- Bad, because A1 requires auditing every `routeMemoryOp({type: 'store'})` callsite (`cli memory store` , `session_restore` , plus any internal call); some may pass payload variants beyond what `ToolPayloadMap['memory_store']` currently models, requiring typed-payload widening or a normalisation layer at the dispatch boundary.
- Bad, because the dispatched memory-read handlers grow a `shape_version` branch each — minor handler-code surface that wants discipline to retire when v1 sunsets.
- Bad, because the v1-deprecation ADR is a real obligation — if it never lands, the dual-schema is unbounded and the "Bad" above becomes permanent.
- Neutral, because the intent-fix commits retained from task #100 attempt 2 (`b91b4fd` + `0eacaf6ec`) keep their forward-compat status — they correct the `EmbeddingScorer.embed` API shape regardless of this ADR's outcome.

Confirmation

- **npm run release is the gate for each of A1, A2, A3** — inherited from ADR-0181's strict-exit discipline.
- **A1 confirmation:** unit test in `forks/agentdb/test/archivist/handlers/memory/store-shape-parity.test.ts` writes a payload via both `archivist.dispatch('memory_store')` AND the post-flip `routeMemoryOp({type: 'store'})` and diffs the persisted on-disk shape — equality required. `tsc --noEmit` accepts every `routeMemoryOp` callsite against the (possibly widened) `ToolPayloadMap['memory_store']` .
- **A2 confirmation:** unit tests in `forks/agentdb/test/archivist/handlers/memory/dual-schema.test.ts` cover round-trip for records written under both v1 and v2 across all 4 dispatched read handlers; a record matching neither schema throws (verified per-handler).
- **A3 confirmation:** acceptance returns the 5 previously-failing checks as PASS and the 4 `adr0181-disp-{get,list,search,sunified}` checks as PASS; total ≥ 668/0/9 (the pre-#100 baseline) on the release.
- **Bounded-transition confirmation:** within one release loop of A3 landing, a new ADR opens with `status: proposed` setting the v1 sunset criteria (typically: zero v1 reads observed across N consecutive releases + a migration mechanism for any residual v1 records). Until that ADR exists, ADR-0181 cannot close.

Architecture

- **Flip mechanism — Strategy 1 (facade-internal flip), chosen.** The flip happens inside the `routeMemoryOp` facade itself, not at every caller. Concretely: in `forks/ruflo/v3/@claude-flow/cli/src/memory/memory-router.ts` , the case `'store':` branch (line ~1010 onward) replaces its inline `storage.*` logic with `await archivist.dispatch('memory_store', payload)` ; the case `'get' | 'list' | 'search' | 'search-unified':` branches replace their inline logic with `await archivist.dispatchRead(<tool>, payload)` . The 12 `routeMemoryOp({type: 'store'})` callsites and ~25 read callsites across ~10 files keep calling the facade unchanged; the facade now dispatches through the archivist on their behalf. Subsequent deletion of the facade (turning every caller into a direct `archivist.dispatch` call) is deferred to its own ADR — same disciplined-bounded posture as the `shape_version` v1 sunset (see [§Open Follow-ups](#)).
- **Strategy 2 (per-callsite flip), rejected.** The alternative — delete `routeMemoryOp({type: 'store' | 'get' | 'list' | 'search' | 'search-unified'})` and convert every caller to a direct `archivist.dispatch / dispatchRead` call — touches ~10 files and ~37 callsites in a single ADR.

Rejected because (a) it fights the same activation battle on 37 fronts when 1 will do; (b) the exit gate (identical on-disk shape, 5 failing checks pass) is unaffected by where the dispatch boundary sits; (c) facade deletion is a code-surface deprecation that wants its own life-cycle tracking, symmetric with the v1 on-disk surface deprecation. Strategy 1 preserves the option of Strategy 2 as a follow-on; Strategy 2 forecloses on the smaller-scope path.

- **A1 audit surface (8 files, 12 callsites).** Even under Strategy 1, every `routeMemoryOp({type: 'store'})` callsite is read and type-checked against the (possibly widened) `ToolPayloadMap['memory_store']` before the flip lands — the typed-overload guarantee is what makes Strategy 1 safe.

File	Callsites
<code>cli/src/memory/memory-router.ts</code>	3 internal (lines 2014, 2098, 2305)
<code>cli/src/mcp-tools/memory-tools.ts</code>	1 (line 123)
<code>cli/src/mcp-tools/hooks-tools.ts</code>	2 (lines 91, 932)
<code>cli/src/mcp-tools/session-tools.ts</code>	1 (line 437)
<code>cli/src/commands/memory.ts</code>	2 (lines 120, 1965)
<code>cli/src/commands/performance.ts</code>	1 (line 244)
<code>cli/src/commands/benchmark.ts</code>	1 (line 292)
<code>cli/src/commands/hooks.ts</code>	1 (line 5217)

- **A3 audit surface (~10 files, ~25 callsites).** Same eight files plus `cli/src/mcp-tools/embeddings-tools.ts`, `cli/src/mcp-tools/agentdb-orchestration.ts`, and `cli/src/commands/embeddings.ts`. The 3 internal `memory-router.ts` read fallbacks (lines 1946, 2059, 2166) are through-facade by construction.
- **ToolPayloadMap['memory_store'] shape.** Step 0 of A1 is the callsite audit (the 8-file / 12-callsite surface above). Either widen `ToolPayloadMap['memory_store']` to cover every variant `routeMemoryOp({type: 'store'})` callers pass today, or land a normalisation layer in the facade at the dispatch boundary that drops/coerces unknown fields explicitly (NOT silently). Compile-time enforcement is non-negotiable — `tsc --noEmit` must reject a payload variant the typed map doesn't cover, per ADR-0181 Phase 5's typed-overload guarantee.
- **On-disk shape_version field.** New rich-meta writes produce records carrying `{shape_version: 2, meta: {namespace, key, content, tags, ttl, ...}, ...}`. Legacy records (no `shape_version` field) are read as v1. This is the discriminator — NOT field-presence sniffing.
- **Read-handler projection.** Each of `handlers/memory/{get,list,search,search-unified}.ts` reads `record.shape_version` (defaulting to 1 if absent), then dispatches projection: v1 → top-level field extraction (`record.namespace`, `record.content`, ...); v2 → meta-field extraction (`record.meta.namespace`, `record.meta.content`, ...). A record matching neither (e.g. has `shape_version` but it isn't 1 or 2; or has `shape_version: 2` but no `meta`) throws a structured error — fails loud per `feedback-no-fallbacks`.
- **Cli read flip (Strategy 1 mechanics).** The 4 memory-read cases in `cli/src/memory/memory-router.ts` (`routeMemoryOp({type: 'get' | 'list' | 'search' | 'search-unified'})`) delegate to `archivist.dispatchRead(<tool>, payload)` inside the facade. The original direct-storage code paths inside the facade branches are deleted once A3's acceptance passes (per ADR-0181 Phase 5's "delete original path once delegation is release-verified"). The ~25 caller sites are unchanged.

- **No changes to other write paths.** Memory writes via `archivist.dispatch('memory_store')` (MCP) already produce v2. The non-memory mcp-tool flips landed in Phase 5 are untouched. The cli's non-memory CLI command writes are out of scope for this ADR.

Execution Plan

Sequential. Each step gated on `npm run release`. Steps are small enough to author solo; if a step's audit surface explodes (e.g. A1's callsite audit finds >20 payload variants), it converts to a focused per-callsite swarm wave with the queen running the typed-payload reconciliation.

Step	Scope	Exit gate
A1	Audit every <code>routeMemoryOp({type:'store'})</code> callsite. Widen <code>ToolPayloadMap['memory_store']</code> or land a normalisation layer accordingly. Flip the router to dispatch through <code>archivist.dispatch('memory_store')</code> . Backfill <code>shape_version: 2</code> on the write.	<code>npm run release</code> passes; the store-shape-parity unit test passes; <code>tsc --noEmit</code> accepts every flipped callsite; acceptance count ≥ baseline.
A2	Implement the <code>shape_version: 1 2</code> discriminator in all 4 dispatched memory-read handlers; cover both shapes in unit tests; cover the neither-shape failure mode.	<code>npm run release</code> passes; dual-schema unit tests pass; neither-shape throws.
A3	Flip the 4 cli memory-read cases to <code>archivist.dispatchRead</code> . Delete the original direct-storage code paths once acceptance passes. Open the v1-sunset ADR.	<code>npm run release</code> passes; 5 previously-failing checks PASS + 4 <code>adr0181-disp-*</code> checks PASS; total ≥ 668/0/9; the v1-sunset ADR exists with <code>status: proposed</code> .

Amendments

Amendment: Completion — full A0+A1+A2+A3 payload delivered (2026-05-17, post-A0-swarm)

Status: this ADR's full implementation payload is on Verdaccio at `patch.177 = 672/681 pass, 0 fail, 9 skip_accepted`. The A0+A1+A2+A3 work landed via a combination of this session's two swarms (`adr-0183-a1` , `adr-0183-a0-pathfix`) and pre-existing coercion commits that solved A2 and A3 via a structurally superior path before ADR-0183 was authored.

Definitive completion table:

Step	Status	Source	Notes
A0 (path-detection prerequisite — not in original	✅ landed	<code>forks/ruflo 0f10328dd</code> (this session)	<code>_resolveDatabasePath + _findProjectRoot</code> route through shared <code>_isProjectRoot()</code> predicate using canonical markers (<code>.ruflo-project CLAUDE.md + .claude/ .git/</code>); no longer false-positives on archivist audit-writer's

Step	Status	Source	Notes
ADR; added in §A0 amendment below)			<cwd>/. <code>claude-flow/</code> <code>mkdir</code> side-effect. Patch.177.
A1 (write flip)	✅ landed	<code>forks/agentdb</code> <code>5213fd3</code> + <code>forks/ruflo</code> <code>c480e38fc</code> (this session, reapply after first-pass revert)	<code>case 'store':</code> in <code>memory-router.ts</code> dispatches through <code>archivist.dispatch('memory_store', normalisedPayload)</code> ; handler stamps <code>shape_version: 2</code> ; normalise-at-facade (no <code>ToolPayloadMap</code> widening — audit confirmed every callsite is a strict subset of <code>MemoryStorePayload</code>); ADR-0094 RC-2 idempotency relocated into the handler; 4-case parity test in <code>agentdb</code> .
A2 (read-side dual-shape)	✅ done — via adapter-merge, superior to originally-planned handler-level discriminator	<code>forks/agentdb</code> <code>a72f664</code> ("MemoryRvfAdapter merges top-level entry fields into metadata; handler records widened") + <code>forks/ruflo</code> <code>b2f25cb44</code> ("list envelope surfaces content + tags from widened <code>MemoryListRecord</code> ") — pre-existing	<code>MemoryRvfAdapter.mergeEntryMetadata(entry)</code> produces <code>{namespace, key, content, tags, ...entry.metadata}</code> (spread-at-end means dispatch-written explicit metadata wins; legacy entries get top-level fields filled in). Single merge point at the adapter layer instead of N handler dispatches; uniform across all readers; no <code>shape_version</code> discriminator needed at read time. The cli's list-envelope mapping surfaces the widened fields.
A3 (cli read flip)	✅ done	pre-existing in <code>memory-router.ts</code>	Read cases dispatch through <code>archivist</code> : <code>case 'search':</code> (line 1282 → <code>archivist.dispatchRead('memory_search')</code>), <code>case 'get':</code> (line 1325 → <code>dispatchRead('memory_retrieve')</code>), <code>case 'list':</code> (line 1419 → <code>dispatchRead('memory_list')</code>). Header comment cites ADR-0181 task #100 as the landing context. <code>search-unified</code> is not a <code>routeMemoryOp</code> type — it's dispatched directly via <code>mcp-tools</code> .

Acceptance trajectory through this session:

Run	Build	Pass / Fail / Skip	Notes
Baseline (pre-session)	patch.166→.172	668→672 / 0 / 9	Coercion commits (agentdb a72f664 , ruflo b2f25cb44) had already made the 5 task-#100 checks green via the adapter-merge A2 approach.
First A1 attempt (write flip)	patch.174	671 / 1 / 9	adr0069-bug3-persist fails — root cause: A1's audit-writer mkdir side-effect tripped _resolveDatabasePath 's .claude-flow/ -as-project-marker check on next-process cold-start.
Revert	patch.175	672 / 0 / 9	Baseline restored; queen authored council report identifying the path-detection root cause.
A1 reapply (per first tie-breaker)	patch.176	671 / 1 / 9	Same 1 fail (intermediate state).
A0 (path-detection fix on top)	patch.177	672 / 0 / 9	adr0069-bug3-persist flips FAIL → PASS. A0+A1+A2+A3 fully landed; baseline clean.

Council records:

- [docs/council/ADR-0183-a1-report.md](#) — first swarm (adr-0183-a1): A1 audit + implementation + revert/reapply trajectory + §A0 (path-detection fix) appendage.

Open follow-ups (carry forward as their own ADRs, not in this one):

1. **v1-shape sunset ADR** (per §Open Follow-ups #1) — substantially de-risked by the adapter-merge approach: shape_version: 2 is stamped on writes but unused at read time (the adapter normalises regardless). The future ADR can scope to "stop stamping; retire the adapter merge when no v1 records remain" — narrower than originally framed. Not urgent; opens when v1 record migration is desired.
2. **routeMemoryOp facade-deletion ADR** (per §Open Follow-ups #1a) — Strategy 2 from §Architecture: per-callsite flip removing the facade entirely. Hygiene-only; the facade-internal dispatch (Strategy 1) is functioning. Opens when the codebase wants the architectural cleanup.
3. **Pipeline-owner residuals** flagged by the first-swarm council report — lockstep-pin warnings (pre-existing M package.json malformed pins), <anonymous_script>:5 JSON parser bug at release-log tail. Not ADR-0183's scope; tracked for the pipeline owners.

ADR-0181 unblock. [ADR-0181 §Closure plan amendment](#) Phase B (CF#8 memory-read handler readiness), Phase G (bench re-baseline), and Phase H (heavy-skip review) are all unblocked now that A3 has been definitively confirmed landed. ADR-0181's "definition of done" should be updated to reflect ADR-0183 closing.

Process note. The first-swarm queen's revert was the structurally correct call against her prompt's strict reading; my first tie-breaker (accept the 1 fail) and second tie-breaker (stand down for replan) both pulled her in different directions. The actual right call was the path-detection root-cause fix she traced and documented — landing A0 cleanly resolved the symptom without needing the plan revision I proposed (bundle A1+A2+A3 atomically, which was based on a wrong shape-divergence diagnosis). The replan amendment was reverted and replaced with the A0

amendment as the canonical record. Lesson: **trust the trace agent's root cause analysis over the prompt-author's a-priori diagnosis** (`feedback-trace-before-hypothesis` applies to the human assistant too).

Amendment: A0 path-detection prerequisite + strategic re-scoping (2026-05-17, post-first-swarm)

This amendment supersedes an earlier draft amendment (titled "Plan revision — A1/A2/A3 fold into atomic A1") that diagnosed the first-swarm failure as a write/read shape divergence and proposed bundling A1+A2+A3 atomically. That diagnosis was wrong. The actual root cause is path-detection side-effects (below); the original A1/A2/A3 split is structurally sound once the prerequisite lands.

First-swarm outcome. `adr-0183-a1` (queen + DA + explorer; explorer/DA messages didn't reach the queen's in-process mailbox, so the queen authored the audit + DA brief herself) landed A1 cleanly: `forks/agentdb a4a3891` (handler stamps `shape_version: 2 + 4-case parity test`) + `forks/ruflo e1f0ecac0` (`case 'store':` dispatches through `archivist.dispatch('memory_store', ...)` with normalisation at the facade; RC-2 idempotency moved into the handler at `store.ts:155-212` ; post-dispatch re-read for envelope parity). Unit tests, tsc, parity test all green. Patch.174 on Verdaccio.

Release-acceptance returned **671/681 pass, 1 fail, 9 skip_accepted** — one new failure: `adr0069-bug3-persist` (ADR-0069 Bug #3 markerless-cwd memory-persist test). The queen traced manually (no native Agent tool in the in-process teammate context) and reverted both fork commits (`agentdb fec78ad` , `ruflo 38e3c9fad`). Post-revert release: **672/681 pass, 0 fail, 9 skip_accepted** on patch.175 — baseline restored. Council record: [docs/council/ADR-0183-a1-report.md](https://docs.council/ADR-0183-a1-report.md).

Strategic finding: the 5 task-#100 failures are already green at baseline. Per the council report's acceptance table, baseline patch.172 was 672/0/9 BEFORE A1 was attempted, with the 5 previously-failing checks (`adr0069-bug3-persist` , `p8-inv12-mem-full` , `e2e-0059-mem-search` , `e2e-0059-p3-unified-both` , `e2e-0059-p3-dedup`) already PASSING. Earlier coercion commits (`forks/agentdb a72f664` , `forks/ruflo b2f25cb44`) fixed the read-side shape divergence directly in the legacy facade reads, independent of any A1/A2/A3 work. **ADR-0183's correctness payload is therefore largely already shipped via coercion.** What remains is architectural hygiene: complete Phase 5 for memory writes (cli internal write router dispatches through archivist, symmetric with the MCP write paths) and the `shape_version` migration scaffolding for future write-path changes.

Actual root cause of the A1 failure: path-detection side-effect, not shape divergence. Verified by trace + code at `forks/ruflo/v3/@claude-flow/cli/src/memory/memory-router.ts:343-369` :

```
const projectRoot = _findProjectRoot();
const inProject = fs.existsSync(path.join(projectRoot, '.claude-flow'));
```

`_resolveDatabasePath()` uses `.claude-flow/` as the project-membership signal. The archivist's audit-writer at `forks/agentdb/src/archivist/audit-writer.ts:93` creates `<cwd>/.claude-flow/data/` on its first audit write (`fs.mkdir(path.dirname(auditPath), {recursive:true})`).

- Pre-A1: writes never dispatch → audit-writer never runs → `<cwd>/.claude-flow/` never appears in markerless cwnds → `inProject = false` → resolves to per-user `$HOME/.claude-flow/data/memory.rvf` (correct).
- Post-A1: writes dispatch → audit-writer mkdir's `<cwd>/.claude-flow/data/` on first call → next process cold-starts and sees `<cwd>/.claude-flow/` → `inProject = true` → resolves to `<cwd>/.claude-flow/memory.rvf` (empty, distinct from the per-user file the previous process wrote) → `Key not found` .

`.claude-flow/` is a runtime-produced directory (audit logs, state files, data subdir), not a human-authored project marker. The cli's own `[ruflo] No project root marker found` warning lists the canonical markers as `.ruflo-`

project , CLAUDE.md + .claude/ , and .git/ . The fix is to align _resolveDatabasePath 's membership check with that canonical set.

Revised plan — A0 prerequisite, then original A1/A2/A3 as written.

Step	Scope	Exit gate
A0 (prerequisite)	One isolated fork commit (~5 LoC) to forks/ruflo/v3/@claude-flow/cli/src/memory/memory-router.ts : change _resolveDatabasePath 's inProject check from fs.existsSync('<projectRoot>/.claude-flow') to a function that tests for the canonical project markers (.ruflo-project OR CLAUDE.md + .claude/ OR .git/) — the same set already referenced by the cli's own warning message. Add a unit test that exercises a markerless cwd with a runtime-created .claude-flow/ dir and asserts the per-user path is still chosen.	npm run release passes; baseline 672/0/9 unchanged; markerless-cwd unit test passes.
A1 (retry)	Original A1 scope as written in §Execution Plan above. The first-swarm implementation (a4a3891 + e1f0ecac0) is a sound starting point — re-apply with any DA-residual-concern adjustments from the council report.	Original A1 exit gate: parity unit test + tsc + npm run release passes; acceptance count ≥ baseline.
A2 (as planned)	Original A2 scope. With A0 in place and A1 re-landed, the read-side dual-schema discriminator on the archivist handlers proceeds as written.	Original A2 exit gate.
A3 (as planned)	Original A3 scope: cli read flip + deletion of legacy facade branches + v1-sunset ADR.	Original A3 exit gate.

The original A1/A2/A3 split in §Execution Plan is the implementation target. A0 is the un-listed prerequisite that the first swarm surfaced.

Carry-forwards from the first swarm (intact, valid input to the A1-retry swarm):

- Audit table — 12 callsites × 8 files, payload-key inventory (council report §Audit findings).
- Strategy decision — **normalise at facade**, not widen ToolPayloadMap['memory_store'] (every callsite's payload is already a strict subset of MemoryStorePayload ; widening adds no value, normalisation drops only the dead-defensive metadata spread at memory-tools.ts:123).
- DA's 3 residual concerns + how the first-swarm implementation addressed each (council report §Strategy decision + §What landed).
- Implementation pattern — facade does await ensureRvfWired() → await getProcessArchivist() → await archivist.dispatch('memory_store', normalisedPayload) → post-dispatch re-read via storage.getByKey for envelope parity.

Open question for A2 (write-path coercion vs new shape_version migration). The earlier coercion commits (a72f664 + b2f25cb44) made the legacy facade reads accept both v1 and v2 shapes. A2's planned dual-schema discriminator on the archivist handlers may overlap with that coercion. Before A2 spawns, audit the coercion pattern

— if it already covers the dispatched-read path, A2's scope shrinks materially or absorbs into A1 (just stamp `shape_version: 2` on writes).

Next swarm composition. A0 is small (1 file, ~5 LoC + 1 unit test) — a 2-agent team is sufficient (queen + implementer; no DA needed for a 5-LoC marker-set replacement). After A0 lands, the A1-retry swarm uses the first-swarm's audit + DA findings as input and is structurally similar to the first swarm.

Open Follow-ups

1. **v1-shape sunset ADR.** A separate future ADR sets the criteria for retiring the v1 read branch (e.g. zero v1 records observed across N consecutive releases + an explicit migration mechanism for any residual v1 records). Tracked here so the dual-schema period is genuinely bounded, not a forever-open compatibility surface.
- 1a. **routeMemory0p facade-deletion ADR.** A separate future ADR sets the criteria for retiring the cli's `routeMemory0p` facade entirely (Strategy 2: every caller becomes a direct `archivist.dispatch / dispatchRead`). Symmetric with the v1-shape sunset — the code-surface deprecation gets its own life-cycle tracking rather than living forever inside the activation ADR. Sequencing: open after A3 lands and acceptance stabilises; the ADR's exit gate is "zero `routeMemory0p({type: 'store' | 'get' | 'list' | 'search' | 'search-unified'})` callsites remain; facade exports removed."
2. **ADR-0181 closure-plan amendment cross-reference.** Once this ADR is `accepted`, the ADR-0181 closure-plan amendment is updated to mark A1/A2/A3 as scoped here; ADR-0181's downstream Phases B + G + H reference this ADR as their unblock condition.
3. **Other non-memory CLI command writes.** This ADR scopes only memory writes. If similar write-path divergence exists for other CLI commands (claims/tasks/agents/swarm direct CLI writes), it surfaces during ADR-0181's downstream phases and gets its own ADR — not folded here.
4. **Multi-process audit under real load.** ADR-0181's Open Follow-up #4 (audit-log behaviour under cli + daemon + hook concurrent dispatch) gets its first real stress test once A1 lands and memory writes route through the archivist from every host process. Findings feed back into ADR-0180 §15 if the single-fd-per-process invariant or the lock-contention budget breaks.

More Information

- [ADR-0181](#) — the activation program this ADR completes the memory-write piece of. See its [§Closure plan amendment](#) (2026-05-17) for the original A1/A2/A3 sequencing this ADR inherits.
- [ADR-0180](#) — the architecture this ADR's activation realises. The dispatch boundary + typed overloads + substrate seam are the prior decisions this ADR depends on.
- [docs/ADR-0181-handover.md](#) — handover snapshot with the full task #100 attempt history, intent-fix retention rationale, and the trace report that identified the write/read field-shape asymmetry as the root cause.
- `feedback-no-fallbacks.md` — the discipline that shapes A2's neither-shape-fails-loud behaviour.
- `feedback-data-loss-zero-tolerance.md` — the shippability bar the 5 failures violate.
- `feedback-trace-before-hypothesis.md` — the process that produced the trace report cited in this ADR's Context.